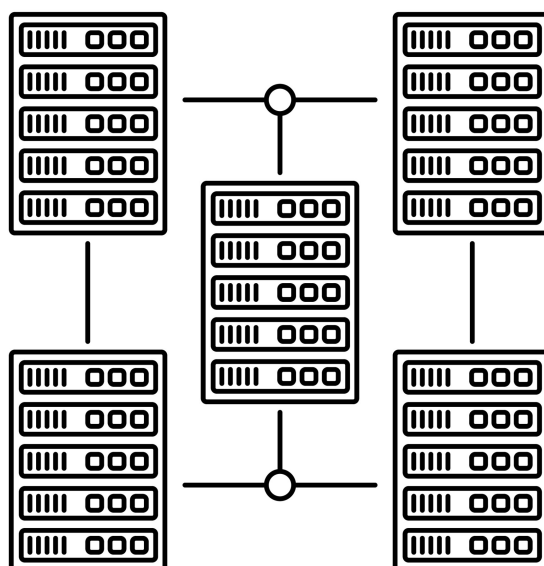


ZERO CONFIG HIGH-AVAILABILITY CLUSTER



Thèse de bachelor présentée par

Jérémie ARCIDIACONO

**Informatique et systèmes de communication avec orientation en
Informatique logicielle**

Août 2026

Professeur-e(s) HES responsable(s)

Florent Glück

Mandant(s) (si existant(s))

ANTS A.I. Systems

Légende et source de l'illustration de couverture:

Ensemble de serveurs interconnectés représentant un système distribué. Source : www.vecteezy.com, ref. URL1

TABLE DES MATIÈRES

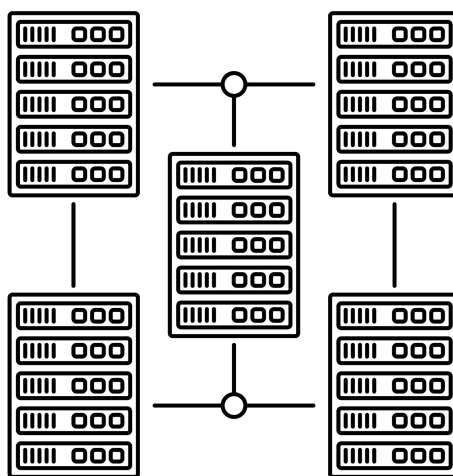
Remerciements	iv
Résumé	v
Liste des acronymes	vi
Liste des figures et des tableaux	vii
Introduction	1
Chapitre 1 : Contexte	3
1.1. Présentation de Kubernetes	3
a) Architecture d'un cluster	4
b) K3s	5
1.2. Présentation de Serf	6
1.3. Présentation des besoins et contraintes	6
Chapitre 2 : Conception	7
2.1. Exigences et fonctionnalités	7
2.2. Architecture générale	8
2.3. ants-os	8
2.4. ants daemon	9
a) Contrôle et supervision	11
b) Bootstrapping	12
c) Cycle de vie d'une machine	15
Chapitre 3 : Implémentation	17
3.1. Structure Go de antsd	17
3.2. Wrapper autour de Serf	17
3.3. Persistance de l'état local	17
Chapitre 4 : Sécurisation	18
Chapitre 5 : Résultats et discussion	19
Conclusion	20
Références documentaires	21

REMERCIEMENTS

Je souhaite remercier M. Florent Glück pour son encadrement et ses conseils tout au long de ce projet. Je remercie également M. Guillaume Chanel pour la création de la feuille de style Typst utilisée pour l'écriture de ce rapport.

RÉSUMÉ

TODO



Candidat-e:

Jérémie Arcidiacono

Filière d'études: ISC

Professeur-e(s) responsable(s):

Florent Glück

En collaboration avec:

ANTS A.I. Systems

Travail de stage soumis à une convention en entreprise: non

Travail soumis à un contrat de confidentialité: non

LISTE DES ACRONYMES

SWIM	Scalable Weakly-consistent Infection-style Process Group Membership
HEPIA	Haute école du paysage, d'ingénierie et d'architecture

LISTE DES FIGURES ET DES TABLEAUX

Fig. 1	Architecture d'un cluster Kubernetes	4
Fig. 2	Diagramme de cas d'utilisation	7
Fig. 3	Architecture d'une machine dans le cluster	8
Fig. 4	Diagramme de composants du daemon ants	10
Fig. 5	Décision au premier démarrage d'une machine	13
Fig. 6	Séquence du mécanisme de bootstrapping	14
Fig. 7	Diagramme de cycle de vie d'une machine	15

RÉFÉRENCES DES URL

URL1	https://www.vecteezy.com/vector-art/17057754-big-data-icon-suitable-for-a-wide-range-of-digital-creative-projects
URL2	https://kubernetes.io/images/docs/components-of-kubernetes.svg

INTRODUCTION

De nos jours, l'engouement pour l'intelligence artificielle engendre un besoin croissant en puissance de calcul ainsi qu'en infrastructures distribuées. Les entreprises cherchent à intégrer ces technologies de pointe pour automatiser leurs processus et améliorer leur productivité. Cependant, elles se heurtent souvent à la complexité technique de leur mise en œuvre, en particulier concernant le déploiement de clusters. Traditionnellement, la mise en place d'un tel environnement est une lourde tâche : elle exige des connaissances pointues, nécessite un temps d'installation conséquent et implique une maintenance continue.

C'est dans ce contexte qu'intervient l'entreprise suisse ANTS A.I. Systems, spécialisée dans la conception et l'offre de solutions d'intelligence artificielle générative. Elle propose une approche novatrice reposant sur trois piliers : des solutions **on-premise** fonctionnant sur des machines basées sur l'architecture ARM vendues par l'entreprise, une architecture **orientée sur la confidentialité** garantissant un contrôle total des données traitées sur site, et enfin un système **Plug-and-Play**. Ce dernier point est crucial : l'infrastructure d'IA doit être entièrement autonome afin de permettre au client final de se passer d'une équipe technique dédiée.

Malheureusement, il s'avère qu'il n'existe actuellement aucune solution permettant de déployer et de maintenir un cluster distribué de manière véritablement « zéro configuration » et clés en main. L'absence d'outils capables de répondre à ces besoins constitue la problématique centrale de ce projet. L'objectif est donc de conceptualiser une nouvelle solution logicielle d'orchestration distribuée.

Ce travail s'inscrit dans le cadre du Travail de Bachelor en Informatique et Systèmes de Communication à la Haute école du paysage, d'ingénierie et d'architecture (HEPIA). Effectué en collaboration avec ANTS A.I. Systems, ce projet se déroule sur une période de 12 semaines, à hauteur de 40 heures par semaine. Il se base sur un travail préparatoire réalisé lors du « projet de semestre »[1], réalisé entre octobre 2025 et avril 2026. Cette base a permis de définir les besoins et les contraintes du projet, et d'identifier les solutions existantes.

La réalisation de ce projet s'est déroulée en plusieurs étapes. Elle a d'abord commencé par la reprise du travail préparatoire du projet de semestre[1], afin de consolider les choix déjà posés et de revoir les besoins du système. Ensuite, une partie importante du travail a consisté à préciser l'architecture cible et à définir le comportement attendu des différents composants.

TODO: modif en fonction de la suite À partir de là, le développement a porté sur la mise en place du logiciel, en particulier antsd — le daemon conçu durant ce travail, qui s'exécute sur chaque machine et automatise la formation puis le maintien du cluster — et son intégration avec Serf et K3s. Cette base a ensuite servi à construire le prototype sur Raspberry Pi, à vérifier le

comportement du système dans des cas simples puis dans des cas de panne. Enfin, une attention particulière a été portée sur la sécurisation du système. Durant toute la durée du projet, des réunions de suivi ont été régulièrement organisées avec le professeur responsable, M. Florent Glück.

La majorité des sources utilisées sont issues de documentations officielles des outils étudiés ainsi que de leurs codes sources publiés sur GitHub. Concernant les illustrations, sauf mention explicite d'une source sous la figure, toutes les figures et tous les diagrammes de ce document ont été réalisés par l'auteur.

Différents modèles d'IA conversationnels (notamment Gemini et Claude via la plateforme Perplexity) ont été utilisés durant ce projet afin de gagner en efficacité. Ils ont été sollicités pour générer de la documentation technique à partir de code source, pour reformuler certains passages de ce mémoire, ou encore pour accélérer le développement logiciel.

Ce document est structuré de la façon suivante : **TODO: modif en fonction de la suite** Le premier chapitre présente le contexte du projet. Il revient sur Kubernetes, K3s et Serf, puis rappelle les besoins et contraintes.

Le deuxième chapitre traite de la conception et de l'architecture. Il décrit les différentes couches du système, le rôle de ants-os, le fonctionnement de antsd, ainsi que le bootstrapping et le cycle de vie d'une machine.

Le troisième chapitre est consacré à l'implémentation. Il explique comment antsd est organisé, etc

Le quatrième chapitre aborde la sécurité. Il présente les limitations et les choix retenus pour protéger le système.

Le cinquième chapitre présente les résultats et la discussion. Il fait le bilan du travail réalisé, met en avant les limites observées et ouvre sur les améliorations possibles.

CHAPITRE 1 : CONTEXTE

Ce premier chapitre présente le contexte dans lequel s’inscrit ce projet, et pose toutes les bases nécessaires à la compréhension de la problématique et de la solution proposée. Il revient sur de nombreuses notions déjà présentées dans le mémoire du projet de semestre[1], qui sont essentielles pour comprendre ce Travail de Bachelor qui en est la continuité.

Il est divisé en plusieurs parties : tout d’abord, une présentation des outils Kubernetes et Serf, qui font partie intégrante de la solution proposée. Ensuite, nous reviendrons rapidement sur différents choix effectués lors du projet de semestre[1]. Enfin, nous présenterons les différents besoins et contraintes identifiés.

1.1. PRÉSENTATION DE KUBERNETES

Kubernetes[2], aussi connu sous le nom de « K8s », intervient en tant qu’orchestrateur de conteneurs open-source. Son rôle est d’automatiser le déploiement, la mise à l’échelle et la gestion d’applications conteneurisées sur un ensemble de machines appelé « cluster ». Kubernetes est aujourd’hui devenu le standard largement adopté par l’industrie.

Outre ses capacités de déploiement, Kubernetes se distingue par ses mécanismes natifs de résilience. Cela justifie son choix pour notre projet qui, par sa nature distribuée, doit être capable de faire face à des pannes matérielles ou des interruptions de réseau.

Le premier pilier de cette robustesse est la gestion de la haute disponibilité. Dans un cluster Kubernetes, la charge de travail n’est pas restreinte à une unique machine. L’orchestrateur permet de répliquer facilement les instances d’une application pour les répartir sur de multiples nœuds. Cette architecture garantit qu’en cas de défaillance matérielle isolée, les requêtes sont automatiquement prises en charge par les répliques fonctionnelles. Le service global de la plateforme ne subit ainsi aucune interruption.

Afin de maintenir cette stabilité sur la durée, le système s’appuie également sur une capacité d’auto-réparation (« self-healing »). Ce mécanisme repose sur la surveillance continue des charges de travail déployées. Lorsqu’un processus s’arrête de manière inattendue, se bloque ou échoue à un bilan de santé, l’orchestrateur intervient de façon autonome. Il se charge d’éliminer le conteneur défaillant et de démarrer immédiatement une nouvelle instance saine, garantissant ainsi que l’état de l’environnement converge en permanence vers l’état désiré.

Enfin, cette vigilance s’applique tout autant à l’infrastructure physique sous-jacente grâce à un mécanisme de surveillance par heartbeats. Chaque machine du cluster signale continuellement son état de santé à l’orchestrateur. Si une machine vient à subir une panne ou une déconnexion et cesse d’émettre ces signaux, le système identifie le nœud comme étant hors service. En réaction,

les charges de travail qui y étaient associées sont immédiatement replanifiées et redémarrées sur les nœuds encore sains. L'ensemble de ces comportements autonomes fait de Kubernetes une solution très pertinente pour gérer une infrastructure distribuée à haute disponibilité.

A) ARCHITECTURE D'UN CLUSTER

Après avoir présenté pourquoi Kubernetes est un choix pertinent, il est nécessaire de présenter brièvement son architecture. Un cluster Kubernetes est composé de deux types de nœuds :

- **Control Plane** (nœud maître) : responsable de la gestion de l'état et de la configuration du cluster.
- **Worker** : responsable de l'exécution des applications.

Dans la Fig. 1, nous pouvons voir un nœud Control Plane qui gère deux nœuds Worker (appelés « Node » sur la figure) et les différents éléments qui les constituent. Nous ignorons volontairement le « Cloud provider API » et « Cloud Controller Manager » qui sont des composants optionnels utilisés dans les environnements cloud, ce qui n'est pas notre cas.

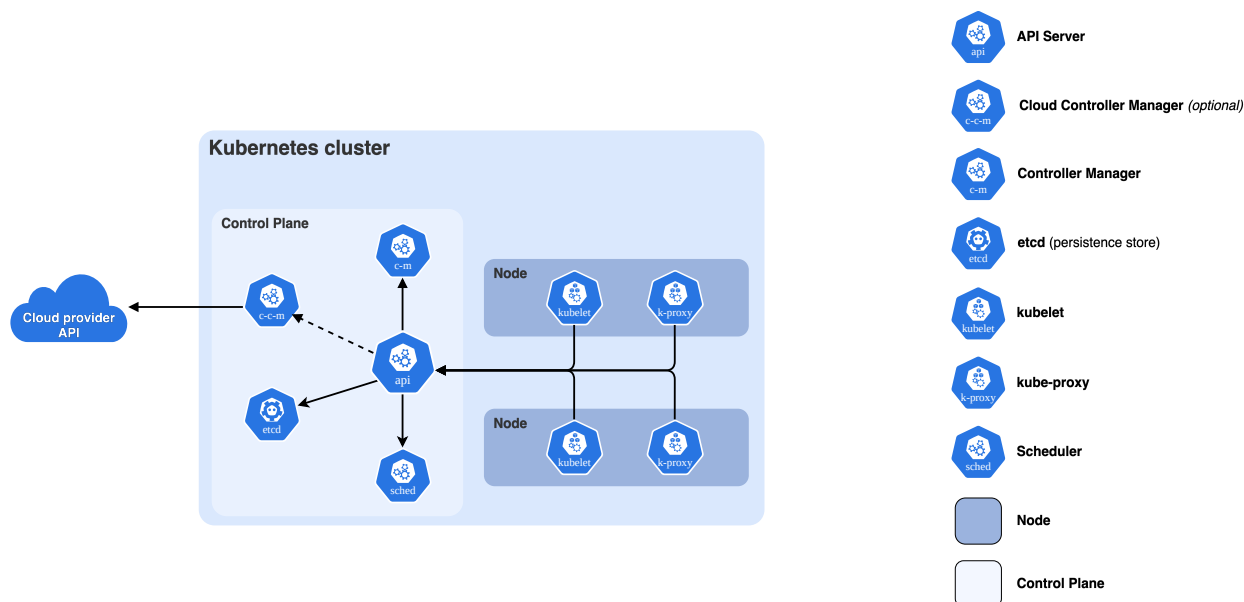


Fig. 1. – Architecture d'un cluster Kubernetes

Source: tiré de kubernetes.io, ref. URL2

Le Control Plane est composé de quatre composants principaux :

- **etcd** : base de données clé/valeur distribuée qui stocke l'état complet du cluster.
- **API Server** : point d'entrée unique pour administrer le cluster ; toutes les communications internes et externes transitent par lui.
- **Scheduler** : détermine sur quel nœud Worker chaque charge de travail doit être déployée.
- **Controller Manager** : surveille en permanence l'état du cluster et effectue les ajustements nécessaires pour maintenir l'état souhaité.

Un nœud Worker est quant à lui composé des éléments suivants :

- **Kubelet** : agent qui reçoit les instructions du Control Plane et gère les charges de travail localement.
- **Kube-Proxy** (optionnel) : assure la gestion du trafic réseau à destination et en provenance des conteneurs.
- **Container Runtime** : moteur d'exécution des conteneurs (par exemple, containerd).

L'unité de déploiement de base dans Kubernetes est le Pod, qui peut contenir un ou plusieurs conteneurs partageant le même espace réseau et de stockage.

B) K3s

K3s[3] est une distribution légère de Kubernetes développée par Rancher (Suse) sous la licence Apache 2.0. Ce point mérite d'être précisé, car il conditionne l'exploitation commerciale de la solution par ANTS A.I. Systems. Cette licence permissive autorise l'utilisation, la modification et la redistribution du logiciel, y compris au sein d'un produit commercial et sans obligation de publier les modifications apportées. Elle accorde en outre une concession explicite de brevets, ce qui limite le risque juridique pour l'entreprise qui l'intègre. Les seules obligations sont de conserver les mentions de licence et de droits d'auteur, et de signaler les fichiers modifiés. Contrairement à une licence copyleft telle que la GPL, elle n'impose donc pas que le produit final soit lui-même distribué en source ouverte.

K3s est conçu pour fonctionner sur des machines aux ressources matérielles limitées, notamment dans des contextes d'edge computing ou d'architectures ARM, tout en restant conforme à l'API Kubernetes standard.

C'est la distribution retenue pour ce projet, en raison de sa faible consommation de ressources, de sa simplicité d'installation et de ses optimisations pour ARM, ce qui correspond aux besoins d'ANTS A.I. Systems.

Dans K3s, la terminologie est légèrement différente de celle de Kubernetes :

- Un **Agent** désigne un nœud Worker.
- Un **Server** représente un nœud Control Plane, mais il intègre également tous les composants d'un Agent et peut donc exécuter des Pods.

Il est possible de former un cluster composé exclusivement de nœuds Server, sans obligation d'avoir des nœuds Agent dédiés. Cela s'explique par le fait qu'un nœud Server est en fait un Agent avec des responsabilités supplémentaires. Il peut donc exécuter des charges de travail tout en assurant la gestion du cluster.

Dans la suite de ce document, nous utiliserons la terminologie de K3s plutôt que celle de Kubernetes.

1.2. PRÉSENTATION DE SERF

Serf[4] est l’outil retenu pour assurer la découverte des nœuds et la communication entre eux. Dans le mémoire du projet de semestre[1], ce sujet est détaillé plus largement dans la section consacrée à Serf. Ici, nous en gardons seulement les éléments utiles à la compréhension de la solution.

Au cœur de Serf, nous retrouvons la bibliothèque Memberlist[5], qui gère le maintien de l’état du cluster via le protocole Scalable Weakly-consistent Infection-style Process Group Membership (SWIM)[6]. Ce dernier assure la détection de pannes et la diffusion des changements d’appartenance par propagation épidémique, c’est-à-dire de proche en proche entre les membres, plutôt que par un point central. C’est ce qui lui permet de rester efficace lorsque le nombre de nœuds augmente, sans introduire de point de défaillance unique. Serf ajoute la couche d’orchestration manquante à Memberlist en complétant le système de gestion de l’appartenance avec la propagation d’événements arbitraires et l’exécution de requêtes/réponses (queries).

Contrairement à Consul, Serf ne s’intéresse pas à l’abstraction de services, mais bien à la gestion des nœuds. Ce point est important pour notre système, dont le rôle est avant tout de gérer l’infrastructure matérielle afin de préparer un environnement stable pour la couche haute, c’est-à-dire K3s.

Serf permet d’abord de découvrir automatiquement les autres machines du réseau local. Pour cela, chaque nœud doit avoir un nom unique, qui peut par exemple être construit à partir de son adresse MAC. Une fois lancé, Serf émet des requêtes multicast sur le réseau local afin de trouver les autres membres du cluster. Cette approche permet une découverte automatique sans configuration d’adresse préalable, ce qui correspond bien à notre objectif zéro-configuration.

Serf propose aussi plusieurs mécanismes de communication entre nœuds. Les tags servent à associer des métadonnées clé/valeur à chaque machine, les événements permettent de diffuser des informations à tout le cluster, et les queries offrent un système synchrone de requête/réponse pour interroger tout ou partie du cluster. Ces mécanismes sont suffisants pour transmettre des informations simples, déclencher des actions et suivre l’état des nœuds.

Enfin, Serf peut conserver l’état du cluster sous forme de snapshots. Cela permet à un nœud qui redémarre de retrouver plus rapidement sa place dans le cluster et d’éviter de repartir de zéro. Pour notre projet, cette capacité est utile, car elle rend la reprise après redémarrage plus rapide et plus robuste.

1.3. PRÉSENTATION DES BESOINS ET CONTRAINTES

TODO

CHAPITRE 2 : CONCEPTION

Le chapitre précédent a permis de définir le contexte général du projet, de revenir sur différents points abordés durant le projet de semestre[1] et de justifier le choix des technologies fondamentales telles que Kubernetes et Serf. Ce deuxième chapitre détaille la conception du système et l'architecture globale retenue pour répondre au cahier des charges. L'objectif est de présenter les mécanismes décisionnels et l'organisation structurelle de la solution avant d'aborder, dans le chapitre suivant, son implémentation technique détaillée.

La conception est basée autour d'un principe simple: l'utilisateur final ne doit pas avoir à connaître la topologie du cluster ni à intervenir sur les nœuds après leur mise sous tension. Le rôle du système consiste donc à automatiser la découverte, le bootstrap, la supervision et l'adaptation du cluster, tout en laissant à l'application finale une interface de contrôle minimale et lisible.

2.1. EXIGENCES ET FONCTIONNALITÉS

Le besoin principal de cette solution réside dans la création d'un environnement à zéro-configuration. Concrètement, le client final doit uniquement brancher physiquement les machines au réseau local et à l'alimentation électrique. Dès cet instant, le système prend le relais de manière totalement autonome pour découvrir les autres membres présents sur le réseau et former un cluster de serveurs fonctionnels. La figure Fig. 2 résume ce scénario, mettant en évidence l'absence d'intervention technique requise avant l'utilisation finale de l'application.

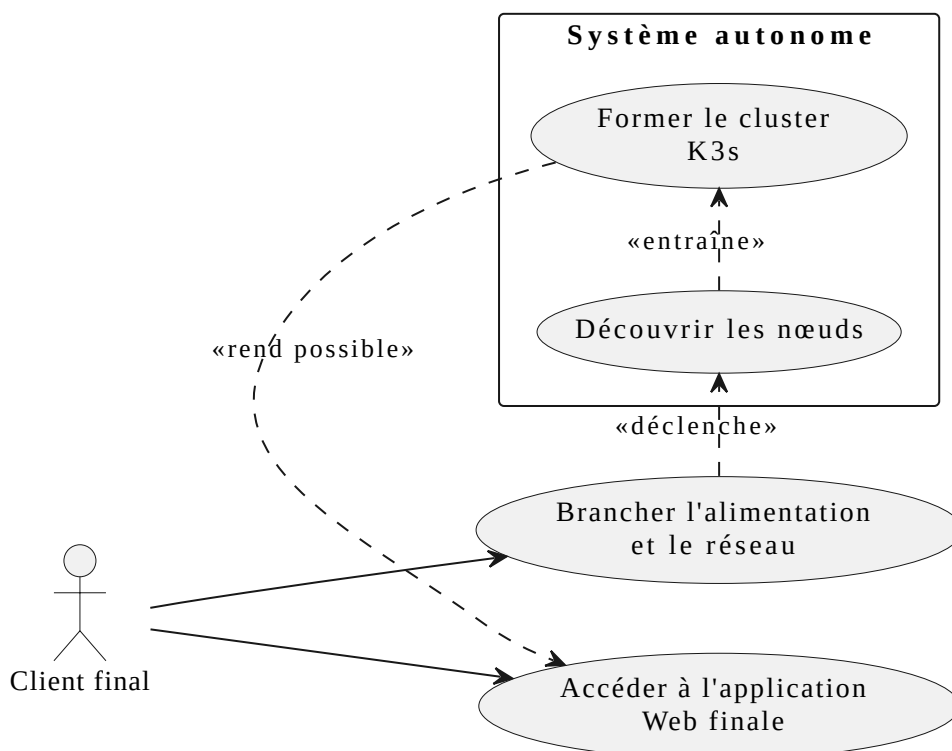


Fig. 2. – Diagramme de cas d'utilisation

La tolérance aux pannes est aussi une contrainte forte. En raison des contraintes strictes liées au consensus de la base de données interne de K3s, le cluster doit garder un nombre impair de servers, entre trois et sept au maximum, afin de garder un quorum stable[7, 8]. Si une machine tombe ou si une nouvelle machine arrive, antsd, le daemon développé dans le cadre de ce projet et qui s'exécute sur chaque machine pour orchestrer le cluster (détaillé dans la section Chapitre 2.4), doit donc pouvoir ajuster le rôle des nœuds sans casser cet équilibre. Un mécanisme de redimensionnement dynamique est prévu afin de promouvoir ou rétrograder des nœuds automatiquement.

2.2. ARCHITECTURE GÉNÉRALE

Afin de répondre aux différents besoins et contraintes énumérés précédemment, nous avons conçu une architecture cible pour notre solution. Pour commencer, partons d'une vue d'ensemble de ce système final. Dans la Fig. 3, nous avons représenté l'une de nos machines en quatre niveaux.

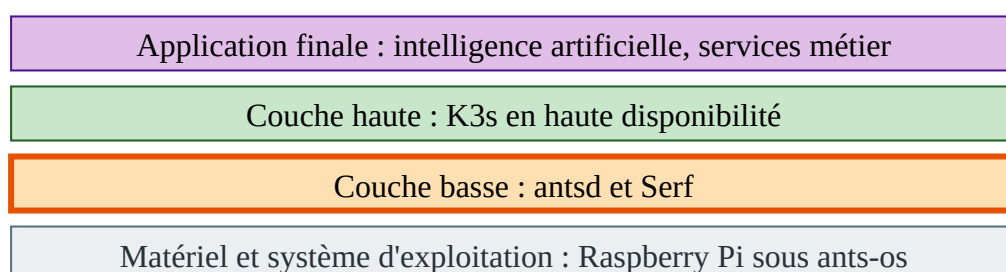


Fig. 3. – Architecture d'une machine dans le cluster

- **Hardware et OS** : la couche la plus basse, elle représente les composants matériels et le système d'exploitation de nos machines physiques. Voir Chapitre 2.3.
- **Couche basse** : composée de ants daemon, qui permet de gérer les machines physiques, de les découvrir, de les provisionner, et de maintenir un état sain au sein du cluster. Voir Chapitre 2.4.
- **Couche haute** : composée de K3s, la distribution Kubernetes choisie (voir Chapitre 1.1).
- **Application finale** : la couche la plus haute. Elle représente l'application métier exploitée par ANTS A.I. Systems. Elle consomme les services exposés par K3s sans se préoccuper du matériel ni de la logique interne de découverte et de maintenance du cluster.

Cette séparation permet d'isoler clairement les responsabilités. Le matériel et le système d'exploitation fournissent une base stable. antsd assure l'orchestration locale et la cohérence du cluster. K3s fournit l'environnement d'exécution des services conteneurisés. Enfin, l'application finale peut se concentrer sur ses propres fonctionnalités métier. Il est important de rappeler que l'application finale n'est pas développée dans le cadre de ce projet.

2.3. ANTS-OS

La base du système est une image ARM64 prête à l'emploi. Pour le PoC, elle cible des Raspberry Pi 5, ce qui permet de tester la solution sur une plateforme simple et peu coûteuse, tout

en restant proche des machines réelles de ANTS A.I. Systems. L'image contient K3s, antsd, les images de conteneurs nécessaires pour un fonctionnement hors ligne, et un service systemd pour lancer antsd au démarrage.

Cette image est construite à l'avance avec HashiCorp Packer[9]. Ce choix évite une installation manuelle sur chaque machine, réduit les différences logicielles entre nœuds et enlève la dépendance au réseau lors du premier démarrage. L'image contient le binaire K3s complet, les images de conteneurs requises pour fonctionner hors ligne, le binaire antsd avec sa configuration, ainsi qu'un service qui lance automatiquement le daemon au démarrage. Les outils de base restent aussi présents pour simplifier le diagnostic lors de la phase de développement.

Le choix de Packer plutôt que d'autres outils tel que rpi-image-gen est motivé par sa capacité à créer des images pour différentes plateformes et architectures.

En pratique, ants-os ne fait pas la logique du cluster. Il prépare simplement une machine propre, stable et identique aux autres, pour que antsd et K3s puissent démarrer de manière fiable.

2.4. ANTS DAEMON

Ants-daemon, aussi appelé antsd, est un daemon Go qui s'exécute sur chaque machine physique du cluster. Il est responsable de la gestion des machines, de leur découverte, de leur provisionnement et de la maintenance d'un état sain au sein du cluster. Il embarque un agent Serf, auquel il délègue la découverte des machines et la communication entre elles. C'est lui qui remplace le rôle humain dans un cluster traditionnel, en automatisant les tâches complexes et manuelles.

La Fig. 4 illustre les composants internes du programme et leurs interactions. On peut par exemple y voir l'agent Serf, auquel une boucle d'événements est attachée pour réagir aux changements de topologie et autres événements du cluster.

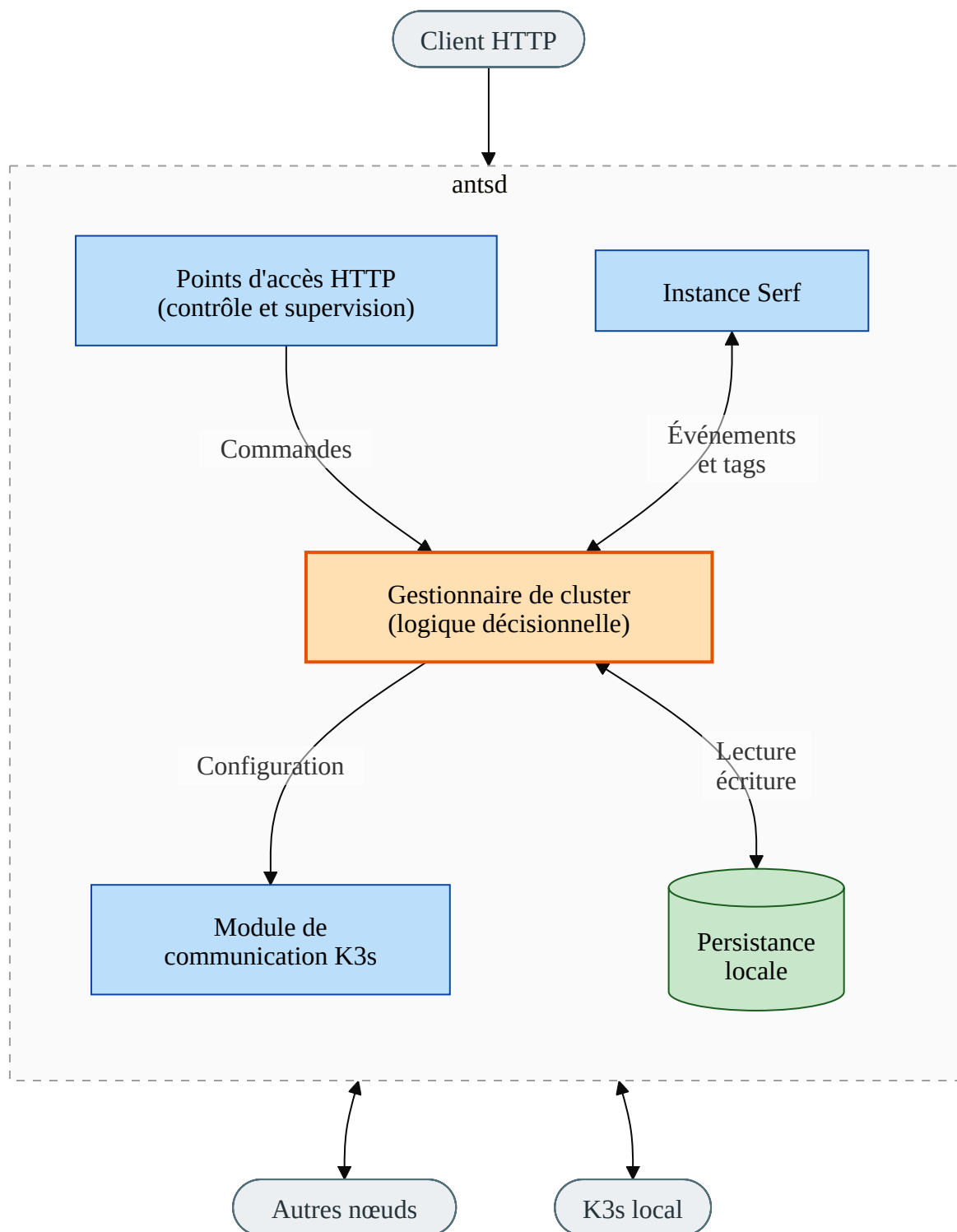


Fig. 4. – Diagramme de composants du daemon ants

Dans cette architecture, antsd joue le rôle de cerveau local. Un gestionnaire de cluster centralise la logique décisionnelle, un module de communication K3s pilote l'instance locale, un wrapper Serf gère les échanges de membership et de diffusion d'événements, et une persistance locale conserve les informations minimales nécessaires à la reprise après redémarrage. Cette séparation permet de limiter le couplage entre les responsabilités, tout en gardant un point d'entrée unique pour les décisions d'orchestration.

Il est important de noter que le daemon interagit avec les autres machines exclusivement via l'agent Serf. Lorsqu'il communique avec une instance de K3s, il interagit toujours avec le processus K3s local, plutôt qu'avec les autres nœuds du cluster. Cela permet de réduire la complexité et d'éviter de dupliquer dans antsd des mécanismes déjà gérés par K3s.

Le choix d'embarquer Serf sous forme de librairie plutôt que comme processus séparé suit la même logique. antsd conserve ainsi la maîtrise du cycle de vie du daemon, des événements et de la communication entre nœuds, sans ajouter une couche d'intégration supplémentaire entre deux programmes distincts.

A) CONTRÔLE ET SUPERVISION

L'utilisateur final a besoin de pouvoir facilement contrôler et surveiller l'état du cluster. Encore une fois, pour suivre la contrainte de simplicité, ces fonctionnalités doivent être intégrées dans la partie « application web finale » de notre architecture. Bien que la réalisation de cette interface web sorte de notre périmètre, il faut néanmoins que nous fournissions les informations nécessaires à son bon fonctionnement. C'est antsd qui est responsable de fournir ces informations, ainsi que de recevoir les commandes de contrôle et de les exécuter sur le système et sur la ou les machines concernées.

Pour l'instant, nous décidons que cela se fera via de simples requêtes HTTP. Certaines permettent de récupérer des informations sur l'état et la santé du cluster, tandis que d'autres permettent d'envoyer des commandes de contrôle, telles que le décommissionnement d'une machine du cluster.

Cette interface reste volontairement minimale. Les besoins identifiés se limitent à quelques points d'accès utiles : un endpoint de statut pour exposer l'état du nœud et du cluster, des points de profilage pour le diagnostic, et une commande de décommissionnement explicite pour retirer proprement un nœud. Ce choix évite de concevoir une API complète alors qu'aucun autre service interne n'a vocation à la consommer.

En pratique, ces points d'accès servent de socle à l'onglet de réglages de l'application web finale. L'utilisateur n'interagit donc pas directement avec antsd pour des opérations complexes : il déclenche une action simple, et antsd traduit ensuite cette demande en opérations sur K3s en s'appuyant sur Serf. Pour accéder à cette interface, l'utilisateur saisit simplement l'adresse IP affichée sur un petit écran présent sur les machines ANTS. Cet affichage local évite de devoir chercher l'adresse du cluster par un autre moyen.

B) BOOTSTRAPPING

Le bootstrapping est la phase qui donne sa forme initiale au cluster. C'est à ce moment que antsd décide si la machine initialise un nouveau cluster ou si elle rejoint un cluster déjà présent sur le réseau local.

Lorsqu'une machine démarre pour la première fois, antsd lance Serf, attend que les autres nœuds deviennent visibles et observe si un cluster existe déjà. Si c'est le cas, la machine rejoint ce cluster et installe K3s avec le rôle attendu. Sinon, elle passe en mode bootstrap et participe à la création du premier cluster K3s.

La Fig. 5 illustre cette première phase de décision. Elle montre comment la machine démarre, observe le réseau local, puis choisit entre rejoindre un cluster déjà formé ou participer au bootstrap initial.

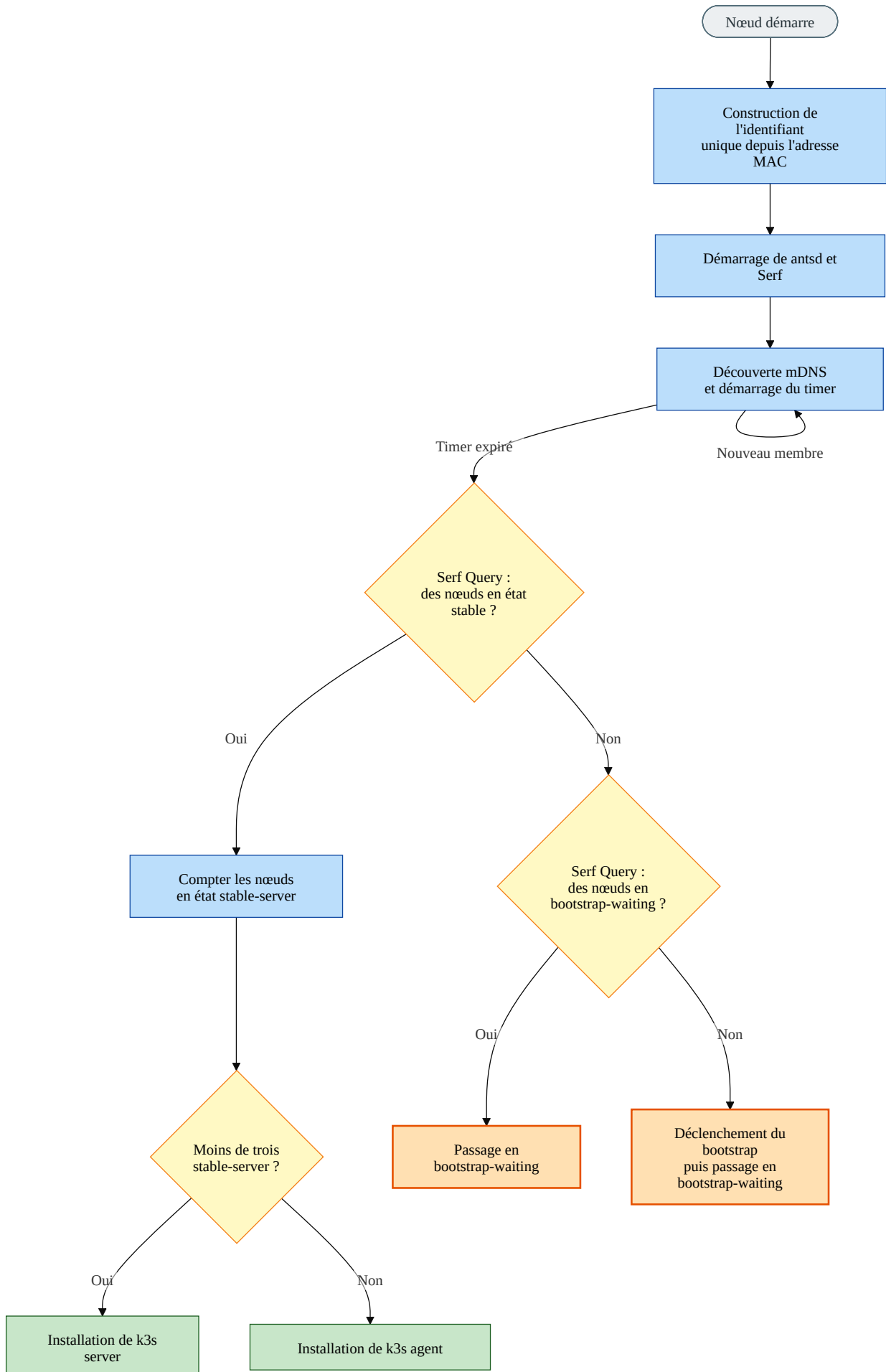


Fig. 5. – Décision au premier démarrage d'une machine

Pour éviter que plusieurs machines créent chacune leur propre cluster, les nœuds passent d'abord par un état d'attente commun. Chacun annonce sa présence, puis un timer local laisse le temps aux autres machines de démarrer. La première machine dont le timer expire devient le nœud initial, noté N0. Elle installe le premier K3s Server puis diffuse l'information aux autres nœuds.

Les autres machines se répartissent ensuite selon leur position dans le groupe découvert. Les nœuds N1 et N2 rejoignent le cluster en tant que K3s Servers afin d'atteindre le quorum minimal du cluster haute disponibilité. Les nœuds suivants rejoignent ensuite le cluster dans le rôle qui convient le mieux à l'état du système.

La Fig. 6 détaille cette deuxième partie du bootstrap. On y voit le passage en mode d'attente, puis la désignation du premier nœud à installer K3s en mode initialisation, avant que les autres machines rejoignent le cluster en parallèle.

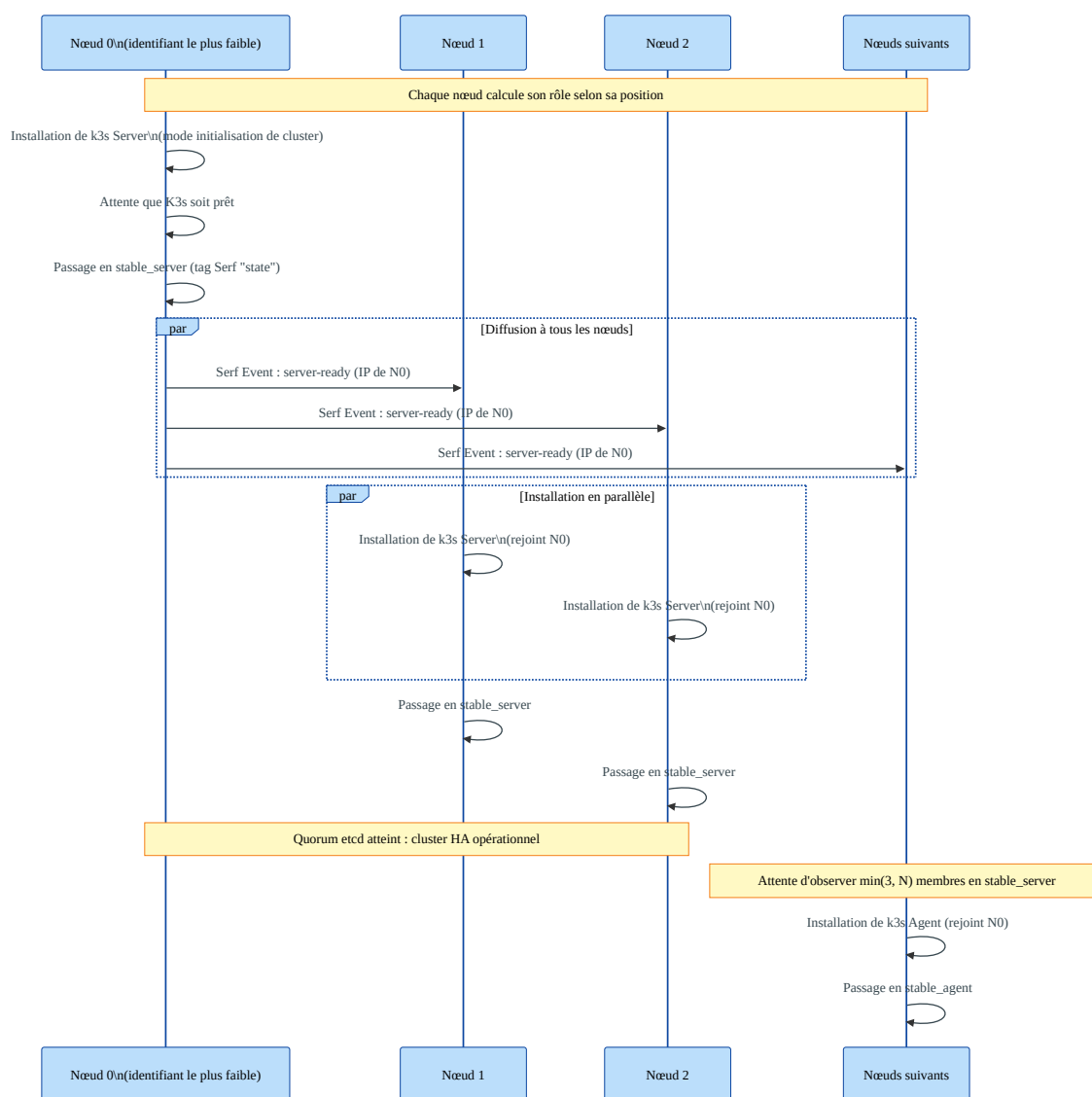


Fig. 6. – Séquence du mécanisme de bootstrapping

Une fois cette phase terminée, antsd enregistre l'état local nécessaire pour retrouver la machine après un redémarrage. Le daemon peut alors reprendre son fonctionnement normal sans refaire tout le processus de départ.

c) CYCLE DE VIE D'UNE MACHINE

Le comportement de antsd tout au long du cycle de vie de la machine est représenté sous la forme d'une machine d'états. La Fig. 7 détaille les différents états possibles et les transitions.

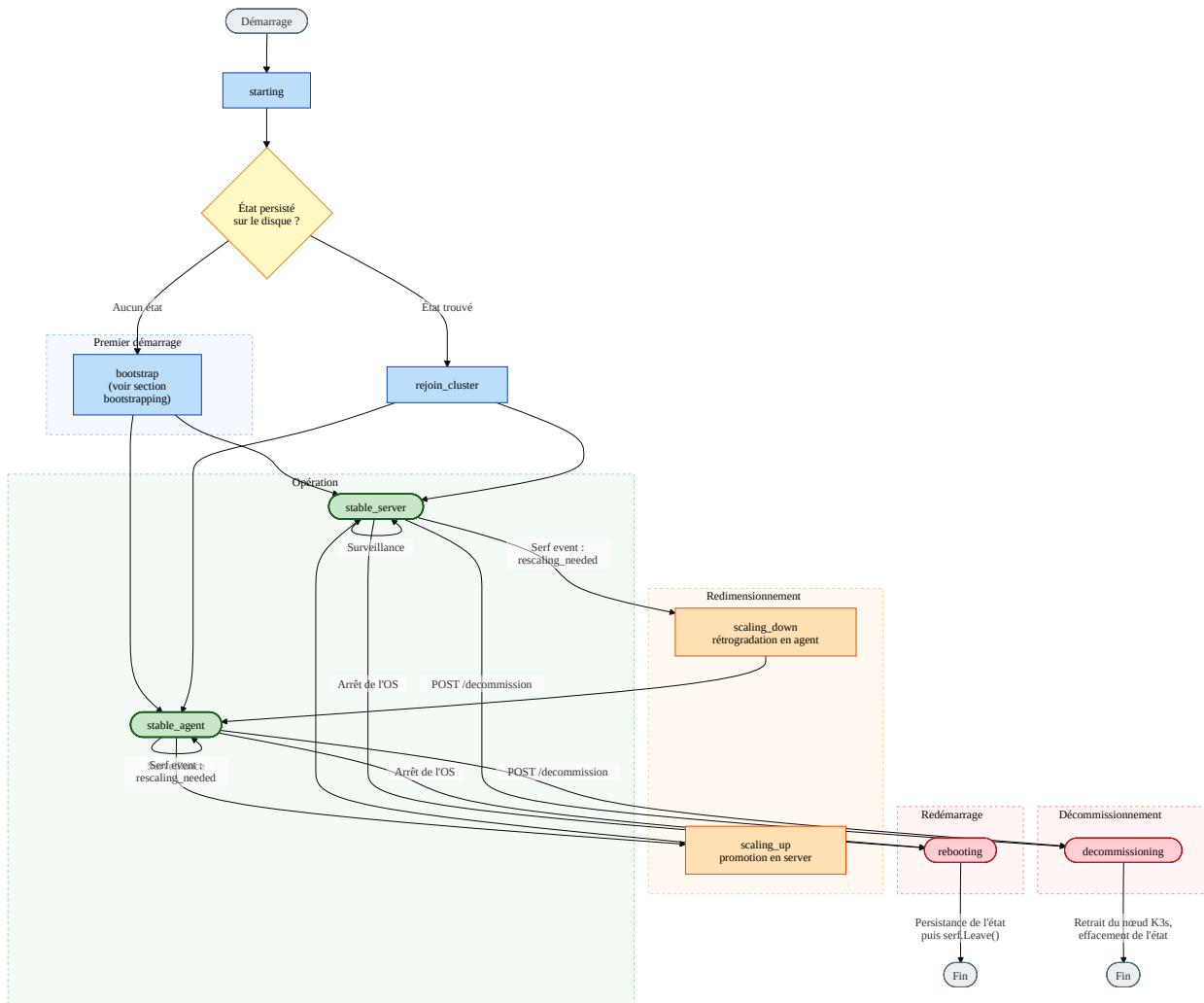


Fig. 7. – Diagramme de cycle de vie d'une machine

Le premier choix distingue un démarrage initial d'un redémarrage connu. Lors d'un premier démarrage, antsd doit déterminer si la machine crée un nouveau cluster ou rejoint un cluster déjà en place. Cette logique, dite de bootstrap, est détaillée dans la section Chapitre b). Lors d'un redémarrage, la présence d'un état local persisté permet au daemon de retrouver rapidement sa place dans le système sans repartir de zéro.

Ensuite, on distingue deux familles d'états : les états stables et les états de transition. Les états stables correspondent aux machines déjà intégrées au cluster K3s et pleinement fonctionnelles. Les états de transition couvrent les opérations de bootstrap, le rescaling, le décommissionnement et la reprise après redémarrage.

Cette séparation évite de mélanger des cas qui ne demandent pas les mêmes actions. Une machine en bootstrap ne doit pas être traitée comme une machine déjà prête, et un nœud en cours de retrait ne doit plus recevoir de nouvelles décisions d’orchestration.

Le rescaling ne se déclenche pas à la moindre variation. Si une panne est courte, K3s peut gérer seul la remise en route normale du nœud. antsd intervient surtout quand la panne dure ou quand l’équilibre du cluster n’est plus bon. Dans ce cas, il peut promouvoir ou rétrograder un nœud depuis l’extérieur du cluster, au lieu de laisser K3s changer le rôle des machines trop tôt. Cette séparation garde le cluster plus stable pendant les redémarrages simples.

Les événements Serf servent enfin à propager ces changements au reste du cluster. La machine d’états s’appuie sur les événements Serf comme mécanisme de propagation. Chaque changement d’état est diffusé vers le reste du cluster, ce qui permet aux autres nœuds d’adapter leur propre comportement sans recourir à des requêtes explicites entre eux. antsd conserve ainsi une vision cohérente.

CHAPITRE 3 : IMPLÉMENTATION

TODO

3.1. STRUCTURE GO DE ANTSD

3.2. WRAPPER AUTOUR DE SERF

3.3. PERSISTANCE DE L'ÉTAT LOCAL

CHAPITRE 4 : SÉCURISATION

TODO

CHAPITRE 5 : RÉSULTATS ET DISCUSSION

TODO

CONCLUSION

TODO

RÉFÉRENCES DOCUMENTAIRES

1. ARCIDIACONO, Jérémie. *SYSTÈME DISTRIBUÉ « ZÉRO-CONFIG » À HAUTE DISPONIBILITÉ*. [en ligne]. Projet de Semestre. Genève, Suisse, 2026. [Consulté le 27 juin 2026]. Disponible à l'adresse: <https://github.com/jeremie-arcidiacono/Zeroconf-Distributed-System>
2. KUBERNETES. Documentation de Kubernetes. [en ligne]. 2026. [Consulté le 24 mars 2026]. Disponible à l'adresse: <https://kubernetes.io/fr/docs/home/>
3. K3S. K3s - Lightweight Kubernetes | K3s. [en ligne]. 20 mars 2026. [Consulté le 24 mars 2026]. Disponible à l'adresse: <https://docs.k3s.io/>
4. HASHICORP. hashicorp/serf. [en ligne]. 2026. [Consulté le 20 juillet 2026]. Disponible à l'adresse: <https://github.com/hashicorp/serf>
5. HASHICORP. hashicorp/memberlist. [en ligne]. 2026. [Consulté le 20 juillet 2026]. Disponible à l'adresse: <https://github.com/hashicorp/memberlist>
6. WIKIPEDIA. SWIM Protocol. [en ligne]. 2026. [Consulté le 20 juillet 2026]. Disponible à l'adresse: https://en.wikipedia.org/wiki/SWIM_Protocol
7. ETCD. etcd FAQ - What is the maximum cluster size?. [en ligne]. [Consulté le 29 juin 2026]. Disponible à l'adresse: <https://etcd.io/docs/v3.6/faq/>
8. K3S. High Availability Embedded etcd | K3s. [en ligne]. 29 juin 2026. [Consulté le 29 juin 2026]. Disponible à l'adresse: <https://docs.k3s.io/datastore/ha-embedded>
9. HASHICORP. hashicorp/packer. [en ligne]. 28 juin 2026. [Consulté le 29 juin 2026]. Disponible à l'adresse: <https://github.com/hashicorp/packer>